The `PostTag` entity defines two `@ManyToOne` associations, as follows:

```
@Entity
@Table(name = "post_tag")
public class PostTag {

    @EmbeddedId
    private PostTagId id;

    @ManyToOne
    @MapsId("postId")
    private Post post;

    @ManyToOne
    @MapsId("tagId")
    private Tag tag;

    private PostTag() {}

    public PostTag(Post post, Tag tag) {
        this.post = post;
        this.tag = tag;
        this.id = new PostTagId(post.getId(), tag.getId());
    }

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTag that = (PostTag) o;
        return Objects.equals(post, that.getPost() &&
                Objects.equals(tag, that.getTag());
    }

    @Override
    public int hashCode() {
        return Objects.hash(post, tag);
    }
}
```

The `@MapsId` annotation instructs Hibernate that the `post_id` and `tag_id` foreign key columns are managed by the `PostTagId` composite identifier, not by the `@ManyToOne` property.

Just like with any bidirectional association, we need to define the helper methods that allow us to keep the `@ManyToOne` and `@OneToMany` sides in sync.

However, as opposed to a standard bidirectional `@OneToMany` association, the `removeTag` helper method in the `Post` entity is much more complex because it needs to locate the `PostTag` associated with the current `Post` entity, as well as the `Tag` object reference in the `tags` collection that needs to be disassociated.

```
public void removeTag(Tag tag) {
    for (Iterator<PostTag> iterator = tags.iterator(); iterator.hasNext(); ) {
        PostTag postTag = iterator.next();
        if (postTag.getPost().equals(this) && postTag.getTag().equals(tag)) {
            iterator.remove();
            postTag.getTag().getPosts().remove(postTag);
            postTag.setPost(null);
            postTag.setTag(null);
            break;
        }
    }
}
```

Notice that, the `removeTag` method relies on the `PostTag` `equals` and `hashCode` methods in order to remove the desired entry.

When rerunning the entity removal example featured in the unidirectional `@ManyToMany` section:

```
post1.removeTag(tag1);
```

Hibernate issues a single delete statement, therefore targeting a single `PostTag` junction record:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 3
```

Because the `@ManyToOne` side manages the foreign key column changes, the internal collection state of the `@OneToMany` side is not taken into consideration unless we remove an entry.

For this reason, reversing the order of the `tags` elements has absolutely no effect:

```
post1.getTags().sort(
    Comparator.comparing(
        (PostTag postTag) -> postTag.getId().getTagId()
    )
    .reversed()
);
```

To materialize the order of elements, the `@OrderColumn` must be used instead:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostTag> tags = new ArrayList<>();
```

When using the aforementioned mapping, rhe `post_tag` junction table features an `entry` column storing the collection element order.

Therefore, if we reverse the element order as we did previously, Hibernate is going to update the `entry` column to maintain the current `PostTag` element order:

```
UPDATE post_tag SET entry = 0 WHERE post_id = 1 AND tag_id = 4
UPDATE post_tag SET entry = 1 WHERE post_id = 1 AND tag_id = 3
```

Mapping the junction entity of a many-to-many table relationship allows us to persist additional column into the junction table (e.g. `created_on`, `created_by`), which cannot be done when using the JPA `@ManyToMany` annotation.

10.7 Hypersistence Optimizer

Knowing which are the most efficient associations is of paramount importance when it comes to designing a high-performance data access layer. But choosing the right mappings is not sufficient. You also have to check the Hibernate configuration properties, as well as the queries that get executed.

For this purpose, I created the Hypersistence Optimizer[1] tool. While you could manually investigate the JPA and Hibernate mappings and configurations, having a tool to do this task on your behalf is going to save you a lot of time.

More, Hypersistence Optimizer can validate your JPA and Hibernate usage during testing, and you could even assert the number of issues detected by the tool and trigger a build failure whenever a performance-impacting issue is detected. This way, you can ensure that future data access logic changes won't affect the performance of your application.

Because you purchased this book, you have a **25$** discount for any Hypersistence Optimizer license. All you have to do is to use the **HO25OFF** coupon code when making the purchase. Or, you can use this link[2] to quickly navigate to the Hypersistence Optimizer sales page with the discount coupon activated.

10.7.1 Testimonials

> "It really pays off when it comes to analyzing complex applications. For architectures that are difficult to manage, it quickly provides clues for further analysis.
>
> This is a huge help both when setting up a new implementation and when optimizing a legacy application."
>
> – Kevin Peters (Software Engineer - codecentric AG)

> "Let's face it, JPA and Hibernate are very powerful but not simple tools. Even experienced developers make mistakes that may result in very heavy database queries, dramatically lowering overall application performance.
>
> Hypersistence Optimizer feels like an additional team member - a JPA expert that is there to help and show how can you optimize your mappings and configurations before you ship them to production."
>
> – Maciej Walkowiak (Freelance Tech Lead)

[1] https://vladmihalcea.com/hypersistence-optimizer/
[2] https://vladmihalcea.teachable.com/p/hypersistence-optimizer/?coupon_code=HO25OFF

11. Inheritance

Java, like any other object-oriented programming language, makes heavy use of inheritance and polymorphism. Inheritance allows defining class hierarchies that offer different implementations of a common interface.

Conceptually, the Domain Model defines both data (e.g. persisted entities) and behavior (business logic). Nevertheless, inheritance is more useful for varying behavior rather than reusing data (composition is much more suitable for sharing structures). Even if the data (persisted entities) and the business logic (transactional services) are decoupled, inheritance can still help varying business logic (e.g. Visitor pattern[1]).

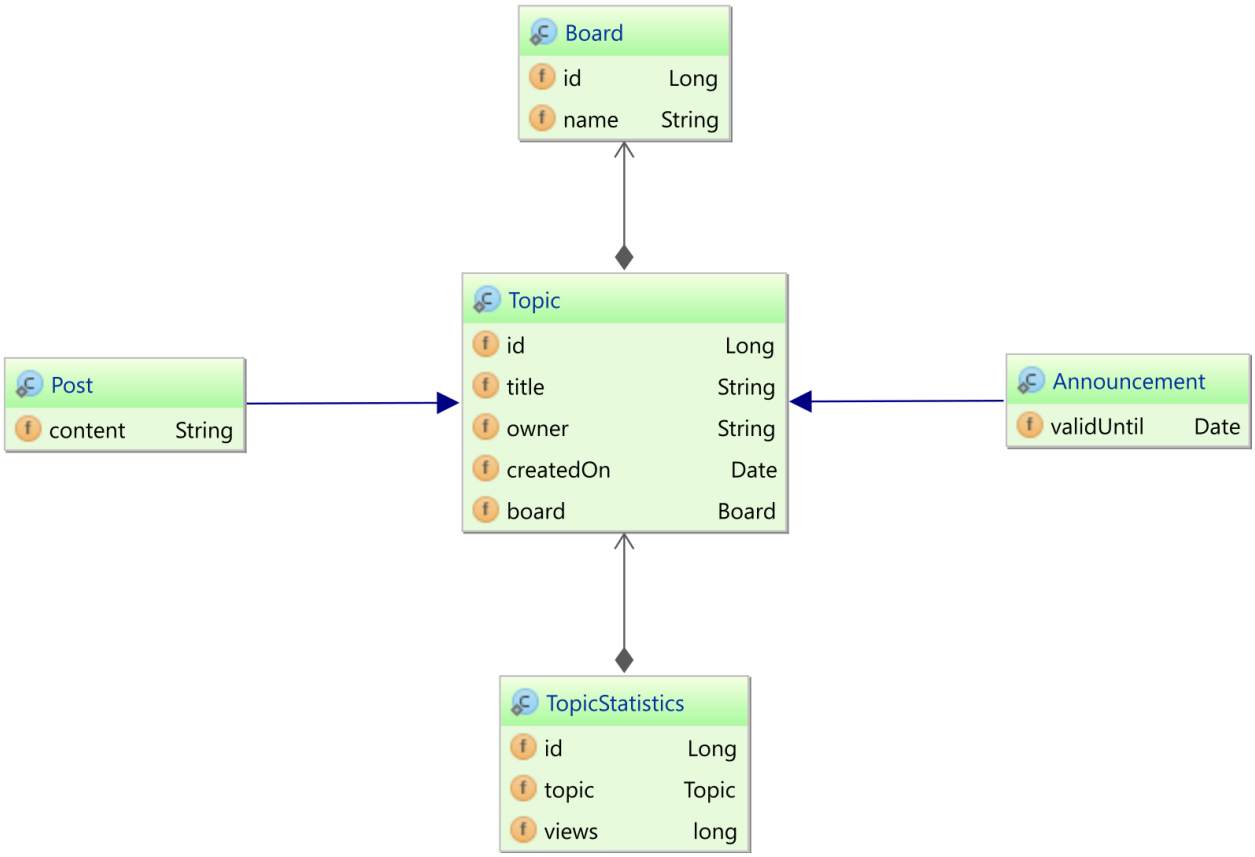

Figure 11.1: Domain Model Inheritance

[1] https://en.wikipedia.org/wiki/Visitor_pattern